6 Understanding the Registers of 8086 Microprocessor

Objective

To study the different types of registers available in the 8086 microprocessor and understand their functions in Assembly Language programming

Introduction

Registers are high-speed storage locations inside the 8086 microprocessor used to temporarily store data, memory addresses, and instructions during program execution. Since registers are located inside the CPU, they provide much faster access than main memory. The 8086 microprocessor contains 14 registers, each 16 bits wide, categorized into four groups: General Purpose Registers, Pointer and Index Registers, Segment Registers, and the Instruction Pointer (IP) along with the Flag Register. Understanding these registers is essential because almost every Assembly Language instruction operates on one or more registers.

Types of 8086 Registers

Register Type	Registers	Purpose
General Purpose	AX, BX, CX, DX	Store data and perform arithmetic and logical operations
Pointer Registers	SP, BP	Store stack and base addresses
Index Registers	SI, DI	Used for indexed addressing and string operations
Segment Registers	CS, DS, SS, ES	Store addresses of code, data, stack, and extra segments
Special Registers	IP, FLAGS	Store instruction address and processor status

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox (if using MASM)
- Text Editor
- Practical Notebook
- 8086 Programming Manual

Procedure

1 Start the computer and open MASM or EMU8086

Create Sticker
Reply to Sweet Bro...

2. Write a simple Assembly Langu age program
3. Load values into different registers
4. Execute the program
5. Observe the register values us ng the debugger
6. Identify the function of each register

Complete Program

```
MODEL SMALL
STACK 100H
CODE
MAIN PROC
MOV AX,1234H
MOV BX,5678H
MOV CX,1111H
MOV DX,2222H
MOV AH,4CH
INT 21H
MAIN ENDP
END MAIN
```

Output

AX = 1234H
BX = 5678H
CX = 1111H
DX = 2222H

Precautions

- Use the correct register names.
- Load valid hexadecimal values.
- Save the program before assembling.
- Check register values after execution.
- Use the debugger carefully.

Viva Voce (Short Question Answers)

Q1. What is a register?

Ans. A register is a high-speed memory location inside the CPU.

Q2. How many registers are available in the 8086 microprocessor?

Ans. Fourteen 16-bit registers.

Q3. Which register is called the accumulator?

Create Sticker
Reply to Sweet Bro...

7 Use of 8086 Instructions

Objective

To understand the basic instructions of the 8086 microprocessor and learn how they are used in Assembly Language programming.

Introduction

An instruction is a command that tells the microprocessor to perform a specific operation. The 8086 microprocessor supports a wide variety of instructions, including data transfer, arithmetic, logical, control transfer, and stack manipulation instructions. Each instruction consists of an opcode and one or more operands. Understanding these instructions is the foundation of Assembly Language programming.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Text Editor

Procedure

1. Open MASM or EMU8086.
2. Write a program using basic instructions.
3. Assemble and link the program.
4. Execute the program.
5. Observe register values.

Complete Program

```
MODEL SMALL
STACK 100H
CODE
MAIN PROC
MOV AX,10
MOV BX,20
ADD AX,BX
SUB AX,5
```

Create Sticker

Reply to Sweet Bro...

```
INC AX
DEC BX
MOV AH 4CH
INT 21H
MAIN ENDP
END MAIN
```

Output

AX = 26

BX = 19

Precautions

- Use valid instructions
- Check syntax carefully
- Verify register values
- Save the source file before assembling
- Execute the program step by step if debugging

Viva Voce (Short Question Answers)

Q1. What is an instruction?

Ans. A command executed by the microprocessor.

Q2. What is the purpose of the MOV instruction?

Ans. To transfer data

Q3. Which instruction performs addition?

Ans. ADD

Q4. Which instruction performs subtraction?

Ans. SUB

Q5. Which instruction increases a value by one?

Ans. INC

Q6. Which instruction decreases a value by one?

Ans. DEC

Create Sticker
Reply to Sweet Bro...

8 Demonstrating MOV Instruction

Objective

To learn how the MOV instruction transfers data between registers and memory locations

Introduction

The MOV instruction is one of the most frequently used instructions in the 8086 microprocessor. It copies data from a source operand to a destination operand without changing the source value. The MOV instruction can transfer data between registers, between memory and registers, and from an immediate value to a register or memory location. It cannot directly transfer data from one memory location to another.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Text Editor

Procedure

1. Create a new Assembly program.
2. Use the MOV instruction to transfer values.
3. Assemble and link the program.
4. Execute the program.
5. Observe the register contents.

Complete Program

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
MOV AX,1234H
MOV BX,AX
MOV CX,BX
MOV AH,4CH
INT 21H
```

Create Sticker

Reply to Sweet Bro...

```
MAIN ENDP
END MAIN
```

Output

AX = 1234H
BX = 1234H
CX = 1234H

Precautions

- Source and destination operands must have the same size.
- Do not use memory-to-memory transfer.
- Verify register contents.
- Save the program before assembling.
- Use valid hexadecimal values.

Viva Voce (Short Question Answers)

Q1. What is the purpose of the MOV instruction?
Ans. To copy data from source to destination.

Q2. Does MOV change the source value?
Ans. No.

Q3. Can MOV transfer memory to memory directly?
Ans. No.

Q4. Which register is commonly used as the accumulator?
Ans. AX.

Q5. Is MOV a data transfer instruction?
Ans. Yes.

Q6. Can MOV load immediate data into a register?
Ans. Yes.

Create Sticker
Reply to Sweet Bro...

6:29

Sent

9 Demonstrating XCHG Instruction

Objective

To study the working of the XCHG instruction for exchanging data between registers.

Introduction

The XCHG (Exchange) instruction swaps the contents of two operands. It is commonly used when programmers need to exchange the values of registers without using an additional temporary register. Both operands must be of the same size. The XCHG instruction is widely used in sorting algorithms and data manipulation programs.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Text Editor

Procedure

1. Open MASM or EMU8086.
2. Write a program using the XCHG instruction.
3. Assemble and link the program.
4. Execute the program.
5. Observe the exchanged register values.

Complete Program

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
MOV AX,1111H
MOV BX,2222H
XCHG AX,BX
MOV AH,4CH
INT 21H
MAIN ENDP
```

Create Sticker

Reply to Sweet Bro...

6:29

Sent


```
END MAIN
```

Output

```
Before Exchange
AX = 1111H
BX = 2222H
After Exchange
AX = 2222H
BX = 1111H
```

Precautions

- Both operands must have the same size.
- Verify register values before and after execution.
- Save the program before assembling.
- Avoid exchanging incompatible operands.

Viva Voce (Short Question Answers)

Q1. What is the purpose of XCHG?

Ans. To exchange data between two operands.

Q2. Does XCHG require a temporary register?

Ans. No.

Q3. Can XCHG exchange two registers?

Ans. Yes.

Q4. Can XCHG exchange different-sized operands?

Ans. No.

Q5. Where is XCHG commonly used?

Ans. In sorting and data manipulation programs.

Q6. Is XCHG a data transfer instruction?

Ans. Yes.

Create Sticker

Reply to Sweet Bro...

6:30

Sent

10 Demonstrating PUSH Instruction

Objective

To understand how the PUSH instruction stores data onto the stack in the 8086 microprocessor.

Introduction

The PUSH instruction places a 16-bit value onto the stack, which is a special memory area used for temporary data storage. When the PUSH instruction is executed, the Stack Pointer (SP) is automatically decremented by 2 because the 8086 stack grows downward in memory. The data is then stored at the new top of the stack. PUSH is commonly used to save register contents before executing subroutines or interrupt service routines.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Text Editor
- Debugger

Procedure

1. Open MASM or EMU8086.
2. Write a program using the PUSH instruction.
3. Assemble and link the program
4. Execute the program
5. Observe the Stack Pointer (SP) and stack contents using the debugger

Complete Program

```
MODEL SMALL
STACK 100H
CODE
MAIN PROC
MOV AX,1234H
MOV BX 5678H
PUSH AX
```

Create Sticker

Reply to Sweet Bro...

6:30
Sent


```
PUSH BX
MOV AH,4CH
INT 21H
MAIN ENDP
END MAIN
```

Output

Before PUSH
SP = FFFEH
After PUSH AX
SP = FFFCH
After PUSH BX
SP = FFFAH
Stack Contents
Top → 5678H
1234H

Precautions

- Initialize the stack before using PUSH.
- Push only 16-bit values in the 8086 microprocessor.
- Avoid stack overflow by ensuring sufficient stack space.
- Observe the Stack Pointer after each PUSH operation.
- Save the program before assembling.

Viva Voce (Short Question Answers)

Q1. What is the purpose of the PUSH instruction?
Ans. It stores data onto the stack.
Q2. Which register points to the top of the stack?
Ans. Stack Pointer (SP).
Q3. How much does SP change after a PUSH instruction in the 8086?
Ans. It decreases by 2 bytes.
Q4. Why is the stack used?
Ans. To temporarily store data, register values, and return addresses.

Create Sticker
Reply to Sweet Bro...

6:30
Sent

11 Demonstrating POP Instruction

Objective

To understand the working of the POP instruction and learn how data is retrieved from the stack in the 8086 microprocessor.

Introduction

The POP instruction is a stack manipulation instruction used to retrieve a 16-bit value from the top of the stack and store it into a specified register or memory location. The stack operates on the Last In, First Out (LIFO) principle. When the POP instruction is executed, the data at the top of the stack is copied to the destination operand, and the Stack Pointer (SP) is automatically increased by 2 bytes. The POP instruction is commonly used to restore register values that were previously saved using the PUSH instruction.

Tools / Components / Instruments

- Computer or Laptop
- MASM (Microsoft Macro Assembler) or EMU8086
- DOSBox (if using MASM)
- Text Editor
- Debugger
- Practical Notebook

Procedure

1. Open MASM or EMU8086.
2. Write the Assembly Language program.
3. Store data on the stack using the PUSH instruction.
4. Retrieve the data using the POP instruction.
5. Assemble and link the program.
6. Execute the program.
7. Observe the register values and Stack Pointer using the debugger.

Complete Program

```
MODEL SMALL
STACK 100H
CODE
```

Create Sticker
Reply to Sweet Bro...

12 Demonstrating PUSHF Instruction

Objective

To study the working of the PUSHF instruction and understand how the Flag Register is stored on the stack.

Introduction

The PUSHF instruction saves the contents of the Flag Register onto the stack. The Flag Register contains status flags and control flags that indicate the results of arithmetic, logical, and control operations. PUSHF is commonly used before executing a subroutine or interrupt to preserve the current flag status. After execution, the Stack Pointer decreases by 2 bytes.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Debugger
- Text Editor

Procedure

1. Open MASM or EMU8086.
2. Write the program using PUSHF.
3. Assemble and link the program.
4. Execute the program.
5. Observe the Flag Register and Stack Pointer.

Complete Program

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
PUSHF
MOV AH,4CH
INT 21H
MAIN ENDP
```

Create Sticker
Reply to Sweet Bro...

END MAIN

Output

Flag Register stored on Stack.

SP decreased by 2 bytes.

Precautions

- Initialize the stack before using PUSHF.
- Observe SP using the debugger.
- Execute the program carefully.
- Ensure there is sufficient stack space.

Viva Voce (Short Question Answers)

Q1. What is the purpose of PUSHF?

Ans. To store the Flag Register on the stack.

Q2. Which register is saved by PUSHF?

Ans. Flag Register.

Q3. How much does SP decrease?

Ans. 2 bytes.

Q4. Why is PUSHF used?

Ans. To preserve processor status before a subroutine or interrupt.

Q5. Which instruction restores the Flag Register?

Ans. POPF.

Q6. Which register contains status flags?

Ans. Flag Register.

Create Sticker

Reply to Sweet Bro...

13 Demonstrating POPF Instruction

Objective

To understand the working of the POPF instruction and restore the Flag Register from the stack.

Introduction

The POPF instruction restores the contents of the Flag Register from the stack. It is generally used after executing a subroutine or interrupt to recover the original processor status. When POPF is executed, the Stack Pointer increases by 2 bytes.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Debugger
- Text Editor

Procedure

1. Write the program using PUSHF and POPF.
2. Assemble the program.
3. Link the object file.
4. Execute the program.
5. Observe the Stack Pointer.

Complete Program

```
MODEL SMALL
STACK 100H
CODE
MAIN PROC
PUSHF
POPF
MOV AH 4CH
INT 21H
MAIN ENDP
END MAIN
```

Create Sticker
Reply to Sweet Bro...

6:31
Sent

Output

Flag Register restored successfully.

SP returned to its original value.

Precautions

- Execute PUSHF before POPF.
- Ensure the stack contains valid flag data.
- Observe Stack Pointer changes.
- Check program syntax carefully.

Viva Voce (Short Question Answers)

Q1. What is the purpose of POPF?

Ans. To restore the Flag Register from the stack.

Q2. Which instruction usually comes before POPF?

Ans. PUSHF.

Q3. How much does SP increase?

Ans. 2 bytes.

Q4. What does POPF restore?

Ans. Processor flags.

Q5. Why is POPF important?

Ans. It restores the previous processor status.

Q6. Which register is affected by POPF?

Ans. Flag Register.

Create Sticker
Reply to Sweet Bro...

14 Demonstrating ADD Instruction

Objective

To perform addition using the ADD instruction in the 8086 microprocessor.

Introduction

The ADD instruction performs arithmetic addition between two operands and stores the result in the destination operand. It updates the Flag Register according to the result. The ADD instruction is widely used for mathematical calculations in Assembly Language programming.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Text Editor

Procedure

1. Open MASM or EMU8086.
2. Write the Assembly Language program.
3. Use the ADD instruction.
4. Assemble and link the program.
5. Execute the program.
6. Observe the result.

Complete Program

```
MODEL SMALL
STACK 100H
CODE
MAIN PROC
MOV AX,15
MOV BX,10
ADD AX,BX
MOV AH,4CH
INT 21H
MAIN ENDP
```

Create Sticker
Reply to Sweet Bro...

6:31
Sent

END MAIN

Output

AX = 25

Precautions

- Use operands of the same size.
- Observe the Carry Flag after addition.
- Verify register values.
- Save the program before assembling.

Viva Voce (Short Question Answers)

Q1. What is the purpose of ADD?

Ans. To add two operands.

Q2. Where is the result stored?

Ans. In the destination operand.

Q3. Does ADD affect flags?

Ans. Yes.

Q4. Which flag indicates carry?

Ans. Carry Flag (CF).

Q5. Can ADD add immediate data?

Ans. Yes.

Q6. Is ADD an arithmetic instruction?

Ans. Yes.

Create Sticker
Reply to Sweet Bro...

6:32

Sent

15 Demonstrating SUB Instruction

Objective

To perform subtraction using the SUB instruction in the 8086 microprocessor

Introduction

The SUB instruction subtracts the source operand from the destination operand and stores the result in the destination operand. It is one of the most frequently used arithmetic instructions in Assembly Language programming. The instruction also updates the processor flags based on the subtraction result.

Tools / Components / Instruments

- Computer or Laptop
- MASM or EMU8086
- DOSBox
- Text Editor
- Practical Notebook

Procedure

1. Open MASM or EMU8086.
2. Write the Assembly Language program.
3. Use the SUB instruction.
4. Assemble the program.
5. Link and execute the program.
6. Observe the result.

Complete Program

```
MODEL SMALL
STACK 100H
CODE
MAIN PROC
MOV AX,25
MOV BX 10
SUB AX BX
MOV AH 4CH
```

Create Sticker

Reply to Sweet Bro...

```
INT 21H
MAIN ENDP
END MAIN
```

Output

AX = 15

Precautions

- Use operands of equal size.
- Observe the Carry and Zero flags.
- Verify the subtraction result.
- Assemble without syntax errors.
- Save the source file before execution.

Viva Voce (Short Question Answers)

Q1. What is the purpose of the SUB instruction?

Ans. To subtract the source operand from the destination operand.

Q2. Where is the result stored?

Ans. In the destination operand.

Q3. Does SUB affect the Flag Register?

Ans. Yes.

Q4. Which flag becomes set if the result is zero?

Ans. Zero Flag (ZF).

Q5. Is SUB an arithmetic instruction?

Ans. Yes.

Q6. Can SUB subtract an immediate value from a register?

Ans. Yes.

Create Sticker
Reply to Sweet Bro...

6:35
1/5,276

Output

Flag Register restored successfully.

SP returned to its original value.

Precautions

- Execute PUSHF before POPF.
- Ensure the stack contains valid flag data.
- Observe Stack Pointer changes.
- Check program syntax carefully.

Viva Voce (Short Question Answers)

Q1. What is the purpose of POPF?

Ans. To restore the Flag Register from the stack.

Q2. Which instruction usually comes before POPF?

Ans. PUSHF.

Q3. How much does SP increase?

Ans. 2 bytes.

Q4. What does POPF restore?

Ans. Processor flags.

Q5. Why is POPF important?

Ans. It restores the previous state.

Documents
Share Edit Delete More